Artificial Intelligence (CS-203)

Objectives:

- ✓ **General Problem Solver (GPS)**
- ✓ **Examples:**
- ✓ **GPS Code in Python**

NOTE: The credit of the python version of GPS goes to Dr. Umair (Associate professor) who worked in BIIT for a very long time. We appreciate his efforts in this regard.

General Problem Solver

The main idea of GPS is to solve a problem using a process called ***means-ends analysis***, where the problem is stated in terms of what we want to achieve at the end. We can solve a problem if we can find some way to eliminate "the difference between what I have (current state) and what I want (goal state and search forward to the goal, or to employ a mixture of different search strategies.

In Python we can refine these notions as follows:

1. We can represent the ***current state*** of the world —"what I have"— or the ***goal state*** –"what I want" – as sets of conditions. We can use lists to implement these states.. Thus, a typical goal might be the list of two conditions (rich famous) and a typical current state might be (unknown poor).

2. We need a list of allowable ***operators***. This list will be constant over the course of a problem, or even a series of problems, but we want to be able to change it and tackle a new problem domain. The list of operators will contain dictionaries whose keys will be
   - ✓ action
   - ✓ preconditions
   - ✓ add
   - ✓ delete

3. An operator can be represented as a structure composed of an ***action***, a list of ***preconditions*** and a list of effects. We can place limits on the kinds of possible effects by saying that an effect either adds or deletes a condition from the current state. Thus, the list of effects can be split into an ***add-list*** and a ***delete-list***.

4. A complete problem is described to ***GPS*** in terms of a starting state, a goal state, and a set of known operators. Thus, GPS will be a function of three arguments. For example, a sample call might be:

   GPS (current_state , goal_state , list-of-operators)

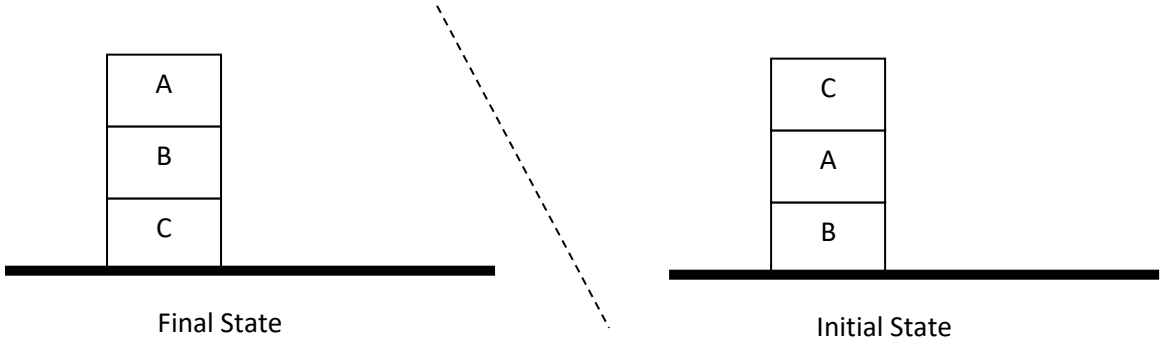
Tracing of GPS function

1. Pick first goal from **goal_state** list.

2. Check it in **current_state** list , it its already in current state the goal is achieved

3. If goal is not in **current_state** list then check it in **add** list, if found in add list of some operator then check its preconditions list, We can apply an operator if we can achieve all the preconditions, so if preconditions are not in **current_state** list then try to achieve those preconditions first.
4. Once the preconditions have been achieved, apply an operator's **add** and **delete** in **current_states** list.

Example 1: Write GPS operators to achieve final state by moving blocks present in the sequence as shown in initial state. Note that at one time only one block can be picked (only from top).




```
block_ops = [
{"action":"Move block C on table",
"preconds":["Block C on A", "C-free"],
"add":["Block C on table", "A-free"],
"delete":["Block C on A"]},
{"action":"Move block A on table",
"preconds":["Block A on B","A-free"],
"add":["Block A on table", "B-free"],
"delete":["Block A on B"]},
{"action":"Move block B on C",
"preconds":["Block B on table", "B-free", "C-free"],
"add":["Block B on C"],
"delete":["Block B on table","C-free"]},
{"action":"Move block A on B",
"preconds":["Block A on table", "A-free","B-free"],
"add":["Block A on B"],
"delete":["Block A on table", "B-free"]}
]

gps(["Block B on table", "Block A on B", "Block C on A", "C-free"],
   ["Block C on table", "Block B on C", "Block A on B", "A-free"]
   ,block_ops )
```

TRACING:

```
0 Achieving: BLOCK C ON TABLE
```

```
1 Achieving: BLOCK C ON A
1 Achieved: BLOCK C ON A
1 Achieving: C-FREE
1 Achieved: C-FREE
0 Action: MOVE BLOCK C ON TABLE   Achieved: BLOCK C ON TABLE
0 Achieving: BLOCK B ON C
1 Achieving: BLOCK B ON TABLE
1 Achieved: BLOCK B ON TABLE
1 Achieving: B-FREE
2 Achieving: BLOCK A ON B
2 Achieved: BLOCK A ON B
2 Achieving: A-FREE
2 Achieved: A-FREE
1 Action: MOVE BLOCK A ON TABLE   Achieved: B-FREE
1 Achieving: C-FREE
1 Achieved: C-FREE
0 Action: MOVE BLOCK B ON C   Achieved: BLOCK B ON C
0 Achieving: BLOCK A ON B
1 Achieving: BLOCK A ON TABLE
1 Achieved: BLOCK A ON TABLE
1 Achieving: A-FREE
1 Achieved: A-FREE
1 Achieving: B-FREE
1 Achieved: B-FREE
0 Action: MOVE BLOCK A ON B   Achieved: BLOCK A ON B
0 Achieving: A-FREE
0 Achieved: A-FREE
EXECUTING MOVE BLOCK C ON TABLE
EXECUTING MOVE BLOCK A ON TABLE
EXECUTING MOVE BLOCK B ON C
EXECUTING MOVE BLOCK A ON B
```

Example 2: A Robot is standing outside a room, the door is closed and the lights are off, Robot can walk. In its mind each action (operator) is store. The final goal is to turn on the lights of the room. Robot will reason that to turn on the lights it has to move inside, but to move inside the door should be opened. So in execution the first step would be the opening of door. Then walking inside and then turning on the lights. We will see how this reasoning process is done in the mind of the Robot, which is using GPS program. Consider following function call of GPS.

gps ([“standing-outside”, “door-closed”, “lights-off”], [“lights-on”], lights-ops)

```
lights_ops = [
{"action":"Turn ON lights",
"preconds":[" inside-room”, “lights-off "],
```

```
"add":[" lights-on "],
"delete":[" lights-off"]},

{"action":"Walk inside the room",
"preconds":[" standing-outside" ,"door-open "],
"add":[" inside-room "],
"delete":[" standing-outside "]},

{"action":" Open the door ",
"preconds":["door-closed"],
"add":["door-opened"],
"delete":["door-closed"]}]
```

Tracing
Do as your assignment

Example 3: Driving son to school

```
sch_ops = [   {
"action": "drive son to school",
"preconds": ["son at home", "car works"],
"add": ["son at school"],
"delete": ["son at home"]
   },
   {   "action": "shop installs battery",
"preconds": ["car needs battery", "shop knows problem", "shop has
money"],
"add": ["car works"],
"delete": []
   },
   {   "action": "tell shop problem",
"preconds": ["in communication with shop"],
"add": ["shop knows problem"],
"delete": []
   },
   {   "action": "telephone shop",
"preconds": ["know phone number"],
"add": ["in communication with shop"],
"delete": []
   },
   {   "action": "look up number",
"preconds": ["have phone book"],
"add": ["know phone number"],
"delete": []
   },
   {   "action": "give shop money",
"preconds": ["have money"],
"add": ["shop has money"],
"delete": ["have money"]
   }
    ]
```

```
gps(["son at home", "have money", "have phone book", "car needs
battery"], ["son at school"],sch_ops )
```

Tracing

```
0 Achieving: SON AT SCHOOL
1 Achieving: SON AT HOME
1 Achieved: SON AT HOME
1 Achieving: CAR WORKS
2 Achieving: CAR NEEDS BATTERY
2 Achieved: CAR NEEDS BATTERY
2 Achieving: SHOP KNOWS PROBLEM
3 Achieving: IN COMMUNICATION WITH SHOP
4 Achieving: KNOW PHONE NUMBER
5 Achieving: HAVE PHONE BOOK
5 Achieved: HAVE PHONE BOOK
4 Action: LOOK UP NUMBER   Achieved: KNOW PHONE NUMBER
3 Action: TELEPHONE SHOP   Achieved: IN COMMUNICATION WITH SHOP
2 Action: TELL SHOP PROBLEM   Achieved: SHOP KNOWS PROBLEM
2 Achieving: SHOP HAS MONEY
3 Achieving: HAVE MONEY
3 Achieved: HAVE MONEY
2 Action: GIVE SHOP MONEY   Achieved: SHOP HAS MONEY
1 Action: SHOP INSTALLS BATTERY   Achieved: CAR WORKS
0 Action: DRIVE SON TO SCHOOL   Achieved: SON AT SCHOOL
EXECUTING LOOK UP NUMBER
EXECUTING TELEPHONE SHOP
EXECUTING TELL SHOP PROBLEM
EXECUTING GIVE SHOP MONEY
EXECUTING SHOP INSTALLS BATTERY
EXECUTING DRIVE SON TO SCHOOL
```

GPS Code

```
def gps(initial_states, goal_states, operators):
   prefix = 'EXECUTING '
for operator in operators:
      operator['action'] = operator['action'].upper()
      operator['add'].append(prefix + operator['action'])
      operator['preconds'] = [pre.upper() for pre in
operator['preconds']]
      operator['delete'] = [dlst.upper() for dlstin
operator['delete']]
      operator['add'] = [add.upper() for add in operator['add']]
initial_states =  [state.upper() for state in initial_states]
goal_states = [goal.upper() for goal in goal_states]
final_states = achieve_all(initial_states, operators, goal_states,
```

```
[])
if not final_states:
return None
actions = [state for state in final_statesif
state.startswith(prefix)]
for a in actions:
print(a)
return actions

def achieve_all(states, ops, goals, goal_stack):
for goal in goals:
      states = achieve(states, ops, goal, goal_stack)
if not states:
return None
   for goal in goals:
if goal not in states:
return None
   return states

def achieve(states, operators, goal, goal_stack):
print(len(goal_stack), 'Achieving: %s' % goal)
if goal in states:
print(len(goal_stack), 'Achieved: %s' % goal)
return states
if goal in goal_stack:
return None
   for op in operators:
if goal not in op['add']:
continue
result = apply_operator(op, states, operators, goal, goal_stack)
if result:
return result

def apply_operator(operator, states, ops, goal, goal_stack):
   result = achieve_all(states, ops, operator['preconds'], [goal] +
goal_stack)
if not result:
return None
print(len(goal_stack), 'Action: %s' % operator['action'], '
Achieved: %s' % goal)
add_list, delete_list = operator['add'], operator['delete']
return [state for state in result if state not in delete_list] +
add_list
```

----------END ----------